

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по рубежному контролю №2

Выполнил:
Студент группы ИУ5-31Б
Паронько Д.И.
Преподаватель:
Гапанюк Ю.Е.

Москва 2025

Листинг кода

database.py

```
from abc import ABC
from dataclasses import dataclass
from typing import List, Any, Callable, Optional
from collections import defaultdict
from functools import wraps

class DatabaseEntity(ABC):
    last_id = 1
    id: int

    def __init__(self):
        self.id = self.__class__.last_id
        self.__class__.last_id += 1

    #метод сброса счетчика id, чтобы модульные тесты были независимы друг от друга
    @classmethod
    def reset_id_counter(cls):
        DatabaseEntity.last_id = 1
        for subclass in cls.__subclasses__():
            subclass.last_id = 1

@dataclass
class Language(DatabaseEntity):
    name: str
    version: str

    def __post_init__(self):
        super().__init__()

@dataclass
class IDE(DatabaseEntity):
    name: str
    language_id: Optional[int] = None

    def __post_init__(self):
        super().__init__()

@dataclass
class LanguageIDE(DatabaseEntity):
    language_id: int
    IDE_id: int

    def __post_init__(self):
```

```

        super().__init__()

# алгоритм нечеткого поиска
def find_levenshtein_distance(str1: str, str2: str):
    dp = [[0 for j in range(len(str1) + 1)] for i in range(len(str2) + 1)]
    for j in range(len(dp[0])):
        dp[0][j] = j
    for i in range(len(dp)):
        dp[i][0] = i

    for i in range(1, len(dp)):
        for j in range(1, len(dp[0])):
            cost = 0
            if str1[j - 1] != str2[i - 1]:
                cost = 1
            dp[i][j] = min(dp[i - 1][j] + 1, dp[i][j - 1] + 1, dp[i - 1][j - 1] +
cost)
    return dp[len(str2)][len(str1)]

# декоратор для печати результата запроса (отчет по запросу)
def print_query(title: str = "Результат запроса", *column_titles):
    def decorator(func: Callable) -> Callable:
        @wraps(func)
        def wrapper(*args, **kwargs):
            print(title)

            result = func(*args, **kwargs)

            column_widths = [0] * len(column_titles)

            for i in range(len(column_titles)):
                max_width = len(column_titles[i])
                for key, values in result:
                    if not isinstance(values, list):
                        values = [values]
                    for value in values:
                        max_width = max(max_width, len(key), len(value))
                column_widths[i] = max_width + 2

            header_parts = [
                f"{column_titles[i]:<{column_widths[i]}}"
                for i in range((len(column_widths)))
            ]
            header_line = " | ".join(header_parts)
            print(header_line)

            print("-" * len(header_line))

            for key, values in result:
                width_key = column_widths[0]

```

```

        width_values = column_widths[1]

        formatted_key = f"{key:<{width_key}}}"
        if not isinstance(values, list):
            values = [values]

        for value in values:
            row_line = []

            row_line.append(formatted_key)

            formatted_value = f"{value:<{width_values}}}"
            row_line.append(formatted_value)

            print(" | ".join(row_line))
        print("-" * len(header_line))
        print("\n")

    return result

return wrapper

return decorator

```

```

class Database:
    def __init__(self):
        self.languages: List[Language] = [] # список ЯП
        self.IDEs: List[IDE] = [] # список IDE
        self.lang_ids: List[LanguageIDE] = (
            []
        ) # список классов Язык-IDE (для связи М:М)

        self.language_map = dict()
        self.ides_map = dict()

    def add_languages(self, langs: List[Language]):
        self.languages.extend(langs)
        for lang in langs:
            self.language_map[lang.id] = lang.name

    def add_ides(self, ides: List[IDE]):
        self.IDEs.extend(ides)
        for ide in ides:
            self.ides_map[ide.id] = ide.name

    def add_lang_ide_relations(self, rels: List[LanguageIDE]):
        self.lang_ids.extend(rels)

    def first_query(self) -> List[Any]:
        query_list = [

```

```

        (ide.name, self.language_map[ide.language_id]) for ide in self.IDEs
    ]
    return sorted(query_list, key=lambda item: item[0])

def second_query(self) -> List[Any]:
    query_dict = defaultdict(int)
    for ide in self.IDEs:
        query_dict[self.language_map[ide.language_id]] += 1
    return sorted(query_dict.items(), key=lambda item: item[1], reverse=True)

def third_query(self, ide_name_prefix: str):
    query_dict = defaultdict(list)

    for el in self.lang_ides:
        ide_name = self.ides_map.get(el.IDE_id)
        if not ide_name: continue

        levenshtein_distance = find_levenshtein_distance(
            ide_name_prefix.lower(), ide_name.lower()
        )

        similary_ratio = (
            (len(ide_name) + len(ide_name_prefix)) - 2 * levenshtein_distance
        ) / (len(ide_name) + len(ide_name_prefix))

        if similary_ratio >= 0.5:
            query_dict[(ide_name, levenshtein_distance)].append(
                self.language_map[el.language_id]
            )

    for el in query_dict.values():
        el.sort()

    sorted_list = sorted(
        query_dict.items(), key=lambda item: item[0][1], reverse=True
    )
    return [(el[0][0], el[1]) for el in sorted_list]

```

unit_test.py

```

import pytest
from database import *

@pytest.fixture
def database():
    test_langs = [
        # ID: 1, 2, 3, 4, 5
        Language("Python", "3.12"),
        Language("Java", "21"),
        Language("Rust", "1.74"),
        Language("Go", "1.21"),
    ]

```

```

    Language("C#", "12"),
    # ID: 6, 7, 8, 9, 10
    Language("JavaScript", "ES2023"),
    Language("TypeScript", "5.2"),
    Language("Swift", "5.9"),
    Language("Kotlin", "1.9"),
    Language("PHP", "8.3"),
]

database = Database()
database.add_languages(test_langs)
return database

@pytest.fixture(autouse=True)
def reset_database():
    DatabaseEntity.reset_id_counter()

def test_first_query(database: Database):
    test_ids = [
        IDE("PyCharm", 1), # Python
        IDE("VS Code", 1),
        IDE("Sublime Text", 1)
    ]

    database.add_ids(test_ids)

    result = database.first_query()
    assert result == [("PyCharm", "Python"), ("Sublime Text", "Python"), ("VS
Code", "Python")]

def test_second_query(database: Database):
    test_ids = [
        IDE("PyCharm", 1), # Python
        IDE("VS Code", 1),
        IDE("Sublime Text", 1),

        IDE("IntelliJ", 2), # Java
        IDE("Eclipse", 2)
    ]

    database.add_ids(test_ids)

    result = database.second_query()
    assert result == [("Python", 3), ("Java", 2)]

def test_third_query(database: Database):
    test_ids = [IDE("PyCharm"), IDE("IntelliJ")]
    database.add_ids(test_ids)

    relations = [LanguageIDE(language_id=1, IDE_id=1), LanguageIDE(language_id=2,
IDE_id=2)]

```

```
database.add_lang_ide_relations(relations)

result = database.third_query("PyCham")
assert len(result) > 0
assert result[0][0] == "PyCharm"
assert "Python" in result[0][1]
```

Результат выполнения:

```
platform linux -- Python 3.12.3, pytest-9.0.2, pluggy-1.6.0 -- /home/juve/git/BMSTU-PCPL-Labs-2025/RK/RK2/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/juve/git/BMSTU-PCPL-Labs-2025/RK/RK2
collected 3 items
```

```
unit_test.py::test_first_query PASSED [ 33%]
unit_test.py::test_second_query PASSED [ 66%]
unit_test.py::test_third_query PASSED [100%]
```

```
===== 3 passed in 0.03s =====
```